Arreglos: integración y derivación numérica

Capítulo 3 · Arreglos y memoria dinámica

Carlos Pinilla

pinilla.carlos@correounivalle.edu.co

Contenido

1. Arreglos en Fortran y en Python.
2. Integración numérica: trapecio y Simpson.
3. Extrapolación de Richardson e integración de Romberg.
4. Derivación numérica: diferencias finitas y paso óptimo.
5. Taller 7: el oscilador amortiguado y el valor de π.

Continuidad con las semanas anteriores

Semanas 1 a 6

- Cada variable guarda un solo número.
- Una serie o un Monte Carlo acumulan en un escalar dentro de un ciclo.

Semana 7

- Una función muestreada en una malla, $y_i = f(x_i)$, son n números que se procesan juntos.
- Integrar y derivar es operar sobre esos n números.

Arreglos en los dos lenguajes; integración en Fortran; derivación en Python.

Declaración y límites en Fortran

Declaración	Índices
`real(dp) :: y(5)`	1, ..., 5
`real(dp) :: t(0:4)`	0, ..., 4
`allocatable :: x(:)`	al reservar

- El primer índice es 1, salvo que se declare otro.
- `size`, `lbound` y `ubound` informan tamaño y límites.

```
real(dp), allocatable :: x(:), y(:)
read *, n           ! se conoce al ejecutar
allocate(x(n), y(n))
...
deallocate(x, y)    ! libera la memoria
```

Memoria dinámica

Un arreglo `allocatable` no ocupa memoria hasta `allocate`: su tamaño puede depender de un dato leído durante la ejecución.

Operaciones sobre el arreglo completo y secciones

```
h = pi/(n - 1)
x = [( (i-1)*h, i = 1, n )]    ! malla en [0, pi]
y = sin(x)                      ! elemento por
     elemento

print *, sum(y(2:n-1))         ! interiores
print *, sum(y(2:n-1:2))       ! índices 2, 4, ...
print *, sum(y(3:n-2:2))       ! índices 3, 5, ...
```

Con $n = 11$: 6.313752, 3.236068, 3.077684.

Expresión	Elementos
`y(2:n-1)`	2 a $n-1$, ambos incluidos
`y(2:n-1:2)`	2, 4, 6, ...
`sum(y(i:j))`	suma de la sección

Las dos secciones alternadas son

exactamente los dos grupos de pesos de la regla de Simpson.

Índices fuera de los límites

```
allocate(y(5))
y = [( real(i, dp), i = 1, 5 )]
do i = 1, 6            ! un índice de más
   print *, y(i)
end do
```

Sin comprobación: `y(6)` imprime un número sin sentido y el programa continúa.

Con `gfortran -fcheck=bounds`:

```
Fortran runtime error: Index '6' of
dimension 1 of array 'y' above upper bound
of 5
```

Fortran no avisa por defecto

Leer fuera de los límites no detiene el programa: el error pasa a los resultados sin que nada lo indique.

Regla del curso

Compilar con `-fcheck=bounds` mientras se desarrolla.

Arreglos como argumentos: forma asumida

```
subroutine malla_uniforme(a, b, x)
  real(dp), intent(in)  :: a, b
  real(dp), intent(out) :: x(:)    ! forma asumida
  integer :: i, n
  n = size(x)                      ! tamaño real
  x = [( a + (i-1)*(b-a)/(n-1), i = 1, n )]
end subroutine malla_uniforme
```

Va en un módulo, como en la Semana 6: el compilador verifica cada llamada.

Se pasa `r(0:4)`; dentro del procedimiento:

Argumento	`lbound`	`ubound`
`t(:)`	1	5
`t(0:)`	0	4

Los límites no viajan

Un argumento `t(:)` siempre empieza en 1. Una tabla con índices desde 0 se declara `R(0:, 0:)`.

Listas y arreglos en Python

```
lista = [1.0, 2.0, 3.0]
lista * 2      # [1.0, 2.0, 3.0, 1.0, 2.0, 3.0]

import numpy as np
arreglo = np.array([1.0, 2.0, 3.0])
arreglo * 2    # [2. 4. 6.]
np.sin(arreglo)   # elemento por elemento
```

- La **lista** es parte del lenguaje: `+` concatena y `*` repite.
- El **arreglo de NumPy** es el equivalente de los arreglos de Fortran: opera elemento por elemento.

Error silencioso

`lista * 2` no multiplica los valores y no da error. Para calcular, `np.array(lista)`.

Índices: base 1 y base 0

Fortran (base 1, final incluido)	Python (base 0, final excluido)	Elementos
`y(1)`	`y[0]`	primero
`y(n)`	`y[-1]`	último
`y(2:n-1)`	`y[1:-1]`	interiores
`y(2:n)`	`y[1:]`	todos menos el primero
`y(1:n-1)`	`y[:-1]`	todos menos el último
`y(2:n-1:2)`	`y[1:-1:2]`	interiores alternados

Al traducir entre lenguajes

El error más frecuente es una rebanada desplazada en un elemento: el programa corre y da un resultado plausible, pero equivocado. Escriba esta tabla para su caso antes de programar.

Asignación, copias y vistas

```
a = np.array([1.0, 2.0, 3.0, 4.0])
b = a               # el mismo arreglo
b[0] = 100.0
print(a)            # [100.  2.  3.  4.]

c = a.copy()        # arreglo nuevo
vista = a[1:3]      # rebanada: vista de a
vista[:] = 0.0      # también cambia a
```

- En Fortran, `b = a` **copia** los valores.
- En Python, `b = a` da otro nombre al **mismo** arreglo.
- Una rebanada es una vista, no una copia.

Para conservar los datos

Use `b = a.copy()` antes de modificar `a`.

Integración numérica

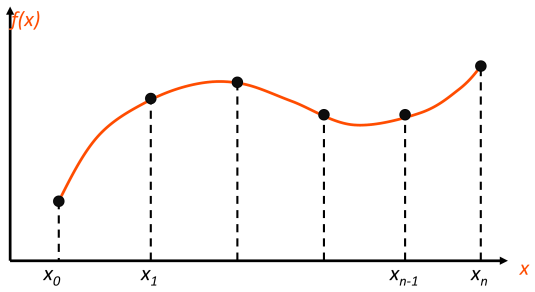
$$I = \int_a^b f(x)\,dx \;\approx\; \sum_{i=0}^{n} c_i\, f(x_i)$$

f: integrando; a, b: límites; x_i: puntos de la malla; c_i: **pesos** de la regla.

Cada regla se distingue por cómo elige los pesos.

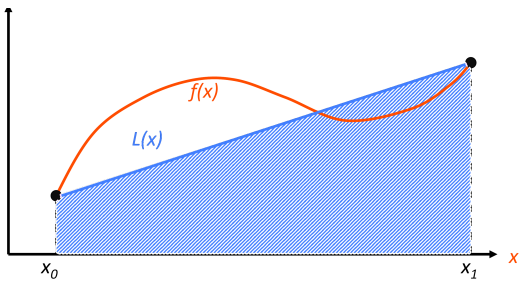
Por qué integrar numéricamente

- La integral analítica no existe en forma cerrada o es impráctica.
- La función solo se conoce en puntos tabulados.



Suma ponderada de los valores de la función en los puntos de la malla.

Regla del trapecio



Recta de Lagrange por (x_0, f_0) y (x_1, f_1):

$$L(x) = \frac{x - x_1}{x_0 - x_1} f_0 + \frac{x - x_0}{x_1 - x_0} f_1$$

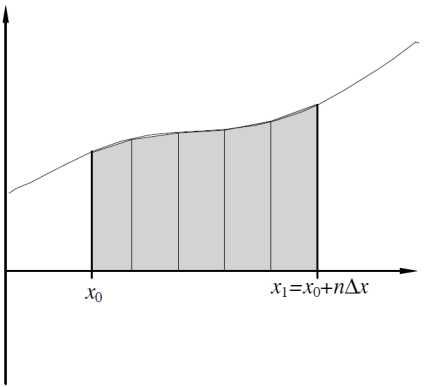
Integrando $L(x)$ entre x_0 y $x_1 = x_0 + h$:

$$\int_{x_0}^{x_1} f \, dx \approx \frac{h}{2} [f_0 + f_1]$$

$$E_{\text{local}} = -\frac{h^3}{12} f''(\xi), \quad x_0 < \xi < x_1$$

El área bajo f se reemplaza por el área del trapecio bajo L.

Regla del trapecio compuesta



N subintervalos de paso $h = (b-a)/N$, puntos $x_i = a + ih$:

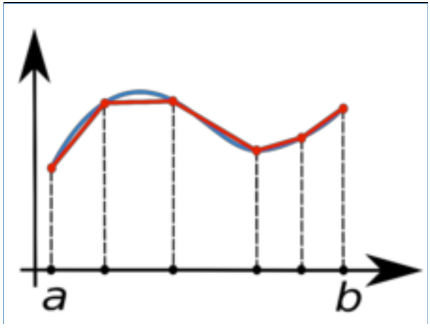
$$T(h) = h\left[\frac{f_0 + f_N}{2} + \sum_{i=1}^{N-1} f_i\right]$$

Cada punto interior pertenece a dos trapecios: peso 1. Los extremos aparecen una vez: peso $1/2$.

En Fortran, con secciones y sin ciclo:

```
h*(0.5_dp*(y(1) + y(n)) + sum(y(2:n-1)))
```

Trapecio con paso variable



Con datos tabulados, cada trapecio tiene su propio ancho:

$$\int_{x_0}^{x_N} f\,dx \approx \frac{1}{2}\sum_{i=1}^{N}(x_i - x_{i-1})\,(f_i + f_{i-1})$$

Los N anchos y las N sumas son diferencias y sumas de dos secciones desplazadas un elemento:

```
0.5_dp*sum((x(2:n) - x(1:n-1))*(f(2:n) + f(1:n-1)))
```

Paso variable

Concentrar puntos donde la función cambia más reduce el error con el mismo número de puntos.

Error local y error global

- Al reducir h a la mitad, el error de **cada** trapecio se divide por 8.
- Pero hay el doble de trapecios: el error **total** se divide por 4.

$$E_{\text{global}} = -\frac{(b-a)\,h^2}{12}\,f''(\xi) \quad \Rightarrow \quad \text{orden 2}$$

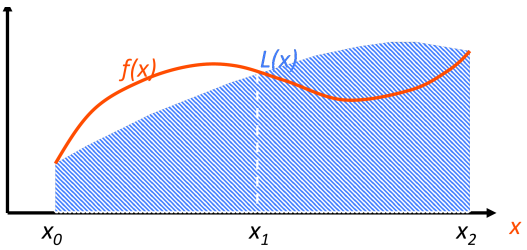
$\int_0^{\pi} \sin x\,dx = 2$: razones 4.03, 4.01, 4.00, 4.00 al duplicar N desde 4 hasta 64.

$$\int_0^4 x\,e^{2x}\,dx = 5216.926477$$

N	Trapecio	Error
1	23847.6639	−357.12 %
2	12142.2245	−132.75 %
4	7288.7877	−39.71 %
8	5764.7621	−10.50 %
16	5355.9471	−2.66 %

Signo negativo: $f'' > 0$ y la recta queda por encima de la curva.

Regla de Simpson 1/3



Parábola $L(x)$ por x_0, x_1, x_2, con paso h.

Integrando la parábola de Lagrange entre x_0 y x_2:

$$\int_{x_0}^{x_2} f\,dx \approx \frac{h}{3}\left[f_0 + 4f_1 + f_2\right]$$

$$E_{\text{local}} = -\frac{h^5}{90} f^{(4)}(\xi)$$

Compuesta, con N **par**:

$$S(h) = \frac{h}{3}\Big[f_0 + 4\sum_{\text{impares}} f_i + 2\sum_{\text{pares int.}} f_i + f_N\Big]$$

Orden 4: al duplicar N, el error cae ≈ 16.

Simpson: relación con el trapecio y regla 3/8

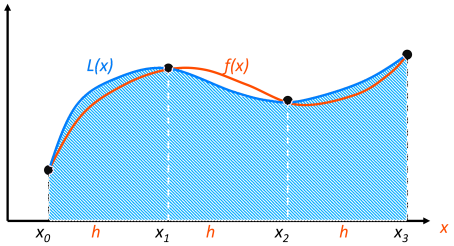
Relación con el trapecio

$$S(h) = \frac{4\,T(h) - T(2h)}{3}$$

Cancela el término h^2 del error del trapecio.

N impar

$\int_0^{\pi} \sin x\, dx$: con $N = 4$, error 4.6×10^{-3}; con $N = 5$ y los mismos pesos, 1.9338: error 6.6×10^{-2}.



$$\int_{x_0}^{x_3} f\, dx \approx \frac{3h}{8}\left[f_0 + 3f_1 + 3f_2 + f_3\right]$$

Cúbica por cuatro puntos: sirve para los tres últimos subintervalos cuando N es impar.

Extrapolación de Richardson

El error del trapecio compuesto es una serie en potencias **pares** de h:

$$T(h) = I + c_1 h^2 + c_2 h^4 + \cdots \qquad T(h/2) = I + c_1 \frac{h^2}{4} + c_2 \frac{h^4}{16} + \cdots$$

I: valor exacto; c_1, c_2: coeficientes que dependen de f y del intervalo, no de h.

Cuatro veces la segunda menos la primera elimina $c_1 h^2$:

$$\frac{4\,T(h/2) - T(h)}{3} = I + O(h^4)$$

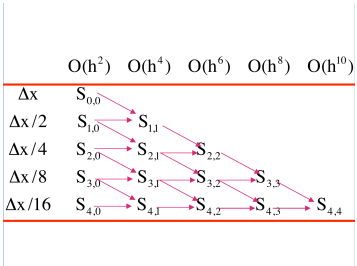
Idea

Dos aproximaciones de orden 2 se combinan en una de orden 4 (Simpson). Repetir el procedimiento elimina h^4, después h^6...

Tabla de Romberg

$$S_{k,0} = T\left(\frac{\Delta x}{2^k}\right), \qquad \Delta x = b - a$$

$$S_{k,j} = \frac{4^j\, S_{k,j-1} - S_{k-1,j-1}}{4^j - 1}$$



a, b: límites; Δx: tamaño del subintervalo en el nivel 0; k: nivel, paso $\Delta x/2^k$; j: columna, orden h^{2j+2}.

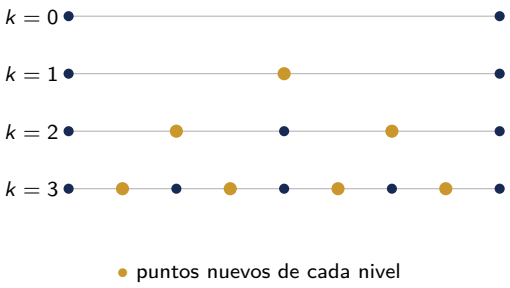
$$\int_1^2 \frac{dx}{x} = \ln 2 = 0.6931471806$$

k	$S_{k,0}$	$S_{k,1}$	$S_{k,2}$	$S_{k,3}$	$S_{k,4}$
0	0.7500000				
1	0.7083333	0.6944444			
2	0.6970238	0.6932540	0.6931746		
3	0.6941219	0.6931545	0.6931479	0.6931475	
4	0.6933912	0.6931477	0.6931472	0.6931472	0.6931472

Con 17 evaluaciones: el trapecio tiene error 2.4×10^{-4}; $S_{4,4}$, 1.3×10^{-9}.

Parada: $|S_{k,k} - S_{k-1,k-1}| < \text{tol}$.

Refinamiento del trapecio sin repetir evaluaciones



Al reducir el paso a la mitad, los puntos anteriores siguen en la malla; solo se agregan los puntos medios:

$$S_{k,0} = \frac{1}{2} S_{k-1,0} + h_k \sum_{\text{puntos nuevos}} f(x)$$

$h_k = \Delta x / 2^k$: paso del nivel k.

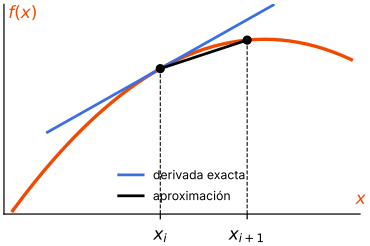
Costo

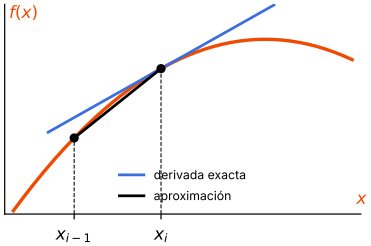
La tabla hasta el nivel k usa $2^k + 1$ evaluaciones de f.

Diferencias adelantada y atrasada

Series de Taylor alrededor de x, con paso $h > 0$:

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(x) + \cdots$$

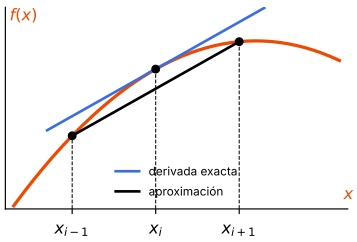




$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2}f''(x) - \cdots \qquad f'(x) = \frac{f(x) - f(x-h)}{h} + \frac{h}{2}f''(x) - \cdots$$

Orden 1: el primer término descartado es proporcional a h; los errores de las dos fórmulas tienen signos contrarios.

Diferencia central y fórmula de cinco puntos



La secante entre x_{i-1} y x_{i+1} es casi paralela a la tangente en x_i.

Central. Restando las dos series, los términos pares se cancelan:

$$f(x+h) - f(x-h) = 2hf'(x) + \frac{h^3}{3}f'''(x) + \cdots$$

$$\Longrightarrow \quad f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6}f'''(x) - \cdots$$

El término en f'' desaparece: **orden 2**.

Cinco puntos. Richardson sobre la central $D(h)$:

$$f'(x) \approx \frac{4D(h) - D(2h)}{3} = \frac{-f_{i+2} + 8f_{i+1} - 8f_{i-1} + f_{i-2}}{12h}$$

Primer término descartado $\propto h^4$: **orden 4**.

Segunda derivada y resumen de órdenes

Central: sumando las dos series,

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} - \frac{h^2}{12} f^{(4)}$$

Adelantada (bordes de la malla):

$$f''(x) \approx \frac{f(x+2h) - 2f(x+h) + f(x)}{h^2}$$

Bordes

La adelantada y la atrasada de f'' son de orden 1, no de orden 2.

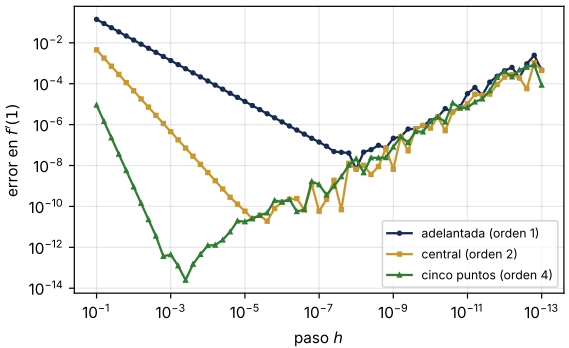
Derivadas de e^x en $x = 1$

Fórmula	Orden	Error, $h = 0.1$	Razón
f' adelantada	1	1.41×10^{-1}	2.03
f' atrasada	1	1.32×10^{-1}	1.97
f' central	2	4.53×10^{-3}	4.00
f' cinco puntos	4	9.07×10^{-6}	16.01
f'' central	2	2.27×10^{-3}	4.00

Razón: error con $h = 0.1$ entre error con $h = 0.05$.

Con el mismo número de evaluaciones, la central tiene un error 30 veces menor que la adelantada.

Error de redondeo y paso óptimo



$$E(h) \approx \underbrace{\frac{h}{2}|f''|}_{\text{truncamiento}} + \underbrace{\frac{\varepsilon\,|f|}{h}}_{\text{redondeo}}$$

$\varepsilon \approx 2.2 \times 10^{-16}$: épsilon de máquina (Semana 2).

	h óptimo	error mín.
adelantada	10^{-8}	7×10^{-9}
central	2.5×10^{-6}	2×10^{-11}

"h más pequeño" no es mejor

Con $h = 10^{-12}$, todas dan errores de $\sim 10^{-4}$.

Derivadas sobre datos tabulados

- El paso es el de la tabla: no se puede reducir.
- Con rebanadas, la central se aplica a todos los interiores a la vez:

```
(f[2:] - f[:-2])/(2*h)
```

- En los extremos solo existen la adelantada y la atrasada.

Un spline cúbico interpola la tabla con continuidad de f, f' y f''; su derivada en 1.10 tiene error 0.009 % *(notebook, sección 5).*

$f(x) = \ln(\tan x)$, $h = 0.05$

x	Fórmula	Error
1.00	adelantada	2.54 %
1.05	central	0.28 %
1.10	central	0.35 %
1.15	central	0.44 %
1.20	atrasada	4.94 %

Taller 7: el oscilador amortiguado

$$x(t) = e^{-\gamma t}\cos(\omega t), \qquad \ddot{x} + 2\gamma\dot{x} + \omega_0^2 x = 0, \qquad \omega_0^2 = \omega^2 + \gamma^2$$

x: posición [m]; $\gamma = 0.5$ s^{-1}: amortiguamiento; $\omega = 2\pi$ rad/s: frecuencia angular; $2\gamma\dot{x}$: fricción por unidad de masa.

Energía por unidad de masa

$$E = \underbrace{\tfrac{1}{2}v^2}_{\text{cinética}} + \underbrace{\tfrac{1}{2}\omega_0^2 x^2}_{\text{potencial}}, \qquad \frac{dE}{dt} = -2\gamma v^2$$

Energía disipada en $[0, T]$

$$\int_0^T 2\gamma v^2\, dt = E(0) - E(T)$$

La integral numérica debe coincidir con la diferencia de energías.

Taller 7: partes y verificación

Parte	Qué se hace
B	Fortran: módulo de Romberg; energía disipada en 3 s con tol 10^{-6}
C	Python: v y a a partir de las posiciones, con cinco fórmulas, frente a las exactas
D	Fortran: el mismo módulo, sin cambios, para $\pi = \int_0^1 \frac{4}{1+x^2}\,dx$; comparación con Monte Carlo

Se entrega el pseudocódigo marco de Romberg con sus tres espacios completos.

Verificación (4 casos)

1. Romberg coincide con $E(0) - E(T)$ y usa exactamente $2^k + 1$ evaluaciones.
2. v y a centrales con $\Delta t = 0.01$ s.
3. Razones 2, 2, 4, 16 y 4 de las cinco fórmulas.
4. π con tol 10^{-5} y 17 evaluaciones.

Sin esqueleto de código; no se permiten funciones de biblioteca que integren o deriven.

Ejercicios de calentamiento

1. **Trapecio con paso variable** (Fortran): 21 puntos tabulados de $\sin x$; comparar con 21 puntos equiespaciados.
2. **Simpson con secciones** (Fortran): $\int_0^4 x\, e^{2x}\, dx$; $N = 2$ da 8240.4114 y las razones tienden a 16.
3. **Deducción de Simpson 1/3** (papel): integrar la parábola de Lagrange.
4. **Derivadas en una tabla** (Python): los cinco puntos de $\ln(\tan x)$, con rebanadas y sin ciclos.

Están en el notebook, en recuadros dorados, con los valores que se deben obtener. No se califican; preparan el Taller 7.

Conclusiones

- Un arreglo guarda los n valores de una malla; las **secciones** operan sobre ellos sin ciclos, en Fortran y en Python, con índices desplazados en uno.
- Las reglas de integración son sumas ponderadas: trapecio de orden 2, Simpson de orden 4.
- La **extrapolación de Richardson** combina aproximaciones para cancelar términos de error: produce Simpson, la tabla de Romberg y la fórmula de cinco puntos.
- En derivación, el error de truncamiento baja con h y el de redondeo sube: existe un **paso óptimo**.
- El orden de un método se verifica con la razón entre errores al reducir h a la mitad.

Próxima semana: máximos, mínimos y ceros de una función.

Referencias

- S. J. Chapman, *Fortran for Scientists and Engineers*, 4.ª ed., McGraw-Hill, 2018, caps. 8 y 9.
- M. T. Heath, *Scientific Computing: An Introductory Survey*, 2.ª ed. revisada, SIAM, 2018, cap. 8.
- R. H. Landau, M. J. Páez y C. C. Bordeianu, *Computational Physics: Problem Solving with Python*, 4.ª ed., Wiley.
- C. Pinilla, *Numerical Methods: Integration* y *Numerical Differentiation*, notas de curso, ICTP.